Language

Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.

Новогодняя скидка 50%
на набор курсов:
Jetpack Compose
Kotlin Coroutines
Dagger + Hilt
Testing

Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 13. Assisted Inject

23 июля 2021

В этом уроке рассмотрим механизм Assisted Inject, который позволяет нам передавать даггеру объекты, которые он не сможет создать сам.

Мы уже рассматривали возможность передачи объекта в компонент (или сабкомпонент), чтобы компонент потом мог использовать этот объект при создании других объектов.

С помощью Assisted injection мы также можем передавать даггеру свои объекты. Только в этот раз мы обойдемся без компонента.

Давайте рассмотрим пример.

У нас есть презентер, который использует объект ServerApi.

```
class MainActivityPresenter @Inject constructor(private val serverApi: ServerApi)
```

Класс ServerApi выглядит так:

```
class ServerApi(val networkUtils: NetworkUtils, val host: String)
```

Ему для создания нужны NetworkUtils и строка с адресом сервера. С объектом NetworkUtils никаких вопросов нет. Даггер знает, как создавать этот объект и сможет его предоставить. А вот с хостом сложнее. Там могут быть разные варианты.

В Уроке 3 мы уже делали похожий пример. Там у нас было пару вариантов серверов и мы использовали @Named, чтобы создавать два разных варианта этого объекта. Но представим, что сейчас у нас немного другая ситуация. На экране есть список серверов и пользователь может выбрать любой из них. А мы должны для выбранного сервера создавать объект ServerApi.

Компонент нам тут никак не поможет. Он создается раньше, чем пользователь выберет сервер на экране, поэтому через него мы передать выбранный сервер не сможем.

Создавать кучу вариантов @Named ServerApi под все возможные серверы, чтобы потом выбрать и использовать нужный - так себе затея.

Нас выручит Assisted Injection. С его помощью даггер может предоставить нам не сам объект ServerApi, а фабрику для его создания. Эта фабрика позволит нам указать хост. А остальные объекты, необходимые для создания ServerApi, даггер подтянет сам (в нашем примере - это NetworkUtils).

Т.е. внутри фабрики даггер так же, как и раньше, будет сам создавать нужный нам объект (ServerApi) и все необходимые для этого объекты (NetworkUtils). А у нас он спросит только значение host.

Давайте внедрять Assisted Injection.

Добавляем аннотацию AssistedInject к конструктору объекта, создание которого мы хотим кастомизировать, т.е. - ServerApi:

```
class ServerApi @AssistedInject constructor(val networkUtils: NetworkUtils, @Assisted val host:
```

Т.е. аннотацию AssistedInject мы используем вместо обычного Inject.

А аннотацией @Assisted мы в конструкторе помечаем тот аргумент, который хотим задавать сами при создании объекта - т.е. host.

Теперь нам надо создать интерфейс, который будет описывать фабрику для создания ServerApi:

```
@AssistedFactory
interface ServerApiFactory {
    fun create(host: String): ServerApi
}
```

Фабрика должна быть помечена аннотацией @AssistedFactory. Она должна содержать метод, который возвращает объект ServerApi, а на вход принимает объект, помеченный нами ранее аннотацией @Assisted, т.е. - host с типом String.

Даггер сам создаст реализацию этого интерфейса и предоставит ее нам. А мы уже используем ее, чтобы создать объект ServerApi со своим хостом.

Фабрика готова. Перепишем конструктор презентера, чтобы получать в него не ServerApi, а его фабрику - ServerApiFactory:

```
class MainActivityPresenter @Inject constructor(private val serverApiFactory: ServerApiFactory)
```

Теперь в презентере мы используем эту фабрику, чтобы создать ServerApi:

```
var serverApi = serverApiFactory.create("dev1.server.com")
```

Мы передаем только host. А даггер сам раздобудет NetworkUtils и создаст нам объект ServerApi.

Однотипные Assisted

Если в конструкторе объекта несколько аргументов одного типа помечено аннотацией @Assisted, то это потребует от нас дополнительных действий.

Смоделируем в нашем примере такую ситуацию. Добавим еще один String аргумент в конструктор ServerApi. Пусть это будет port.

```
class ServerApi @AssistedInject constructor(
    val networkUtils: NetworkUtils,
    @Assisted val host: String,
    @Assisted val port: String)
```

Теперь у нас два String объекта с аннотацией @Assisted.

Фабрика могла бы выглядеть так:

```
@AssistedFactory
interface ServerApiFactory {
    fun create(host: String, port: String): ServerApi
}
```

Но это не сработает. У нас получается два String параметра и даггер сам не разберется, какой String из фабрики в какой аргумент конструктора ServerApi передавать. Нам надо их явно отличать.

Специально для этих целей у аннотации @Assisted есть параметр, где мы можем задать имя:

```
class ServerApi @AssistedInject constructor(
    val networkUtils: NetworkUtils,
    @Assisted("host") val host: String,
    @Assisted("port") val port: String)
```

Эти же имена используем и в фабрике:

```
@AssistedFactory
interface ServerApiFactory {
    fun create(
        @Assisted("host") host: String,
        @Assisted("port") port: String
    ): ServerApi
}
```

Схема похожа на использование аннотации Named.

Теперь мы в презентере используем фабрику, которое передаем два String значения:

```
var serverApi = serverApiFactory.create("dev1.server.com", "80")
```

default

Если нам в фабрике нужны значения по умолчанию, мы можем их использовать. Например, для порта можно поставить значение по умолчанию 80:

```
@AssistedFactory
interface ServerApiFactory {
    fun create(
        @Assisted("host") host: String,
        @Assisted("port") port: String = "80"
    ): ServerApi
}
```

Теперь при использовании фабрики значение порта можно не указывать.

```
serverApi = serverApiFactory.create("dev1.server.com")
```

Назад Вперёд

Комментарии

RE: Урок 13. Assisted Inject — Руслан 16.08.2021 15:50
var serverApi = serverApiFactory.create("dev1.server.com") А где нужно данную строку прописать? Я пробовал и Presenter и во фрагменте. Всегда кидает cannot be provided without an @Provides-annotated method.
Ответить

RE: Урок 13. Assisted Inject — Руслан 16.08.2021 23:05
А все, оказывается версию даггера не ту подключил
Ответить

RE: Урок 13. Assisted Inject — Лют 16.03.2023 12:19
Может глупый вопрос, но всё же. А почему даггер не может различать в двух аргументах, какой из них есть ху. Во первых у нас же есть имена, а во вторых порядок расположения.
Ответить

RE: Урок 13. Assisted Inject — Dmitry Vinogradov 17.03.2023 17:48
Какие то внутренние ограничения. В официальной доке есть упоминание об этом:

"Unfortunately, using parameter names to disambiguate parameters is not possible as there are situations where the parameter names are not kept. Please see this issue (https://github.com/google/dagger/issues/2281) for more information."
Ответить

Обновить список комментариев
RSS лента комментариев этой записи

Добавить комментарий

Тема

Отправить

XComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.